Propeller 2 • Application Note P2AN003

Generate Analog Waveforms and Audio on a P2 Pin

No external DAC — sample playback, synthesis, dithering, and mixing, straight from the smart pin

August 2026

Version 1.0.2

What You'll Build

Turn a single P2 pin into a 16-bit audio DAC — playing samples, synthesizing waveforms, and mixing channels, with no converter chip.

A shared output stage — configure the dithered DAC, pace it from the smart pin's own sample clock — then a small catalog of recipes you choose among by what your project needs:

- **Sample playback (DDS)** — resample a stored buffer to any pitch
- **Waveform synthesis** — sine, sawtooth, and triangle from the phase
- **Dithering** — trade the 8-bit DAC up to clean 16-bit resolution
- **ADC → DAC passthrough** — a live analog wire through the chip
- **Mixing & panning** — sum many voices to a stereo pair of pins
- **The ceiling** — a 32-stream software mixer (reference)

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut, current release • any P2 board, plus an RC filter or a headphone jack to hear the output.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

Every P2 I/O pin contains a DAC. With nothing more than the smart pin and a few lines of code, you can turn one pin into a 16-bit audio-quality analog output — play back a recorded sample, synthesize a waveform, pass a live signal through the chip, or mix several voices to a stereo pair — with no external converter. This note builds the DAC output stage once as a clean, reusable base, then walks a small set of techniques you choose among to play, synthesize, and mix analog signals for whatever your project needs.

What you'll build: a single-pin DAC streaming engine that plays samples and synthesizes waveforms, paced by the smart pin's own sample clock, plus a stereo mixer that sums several voices to two pins.

Prerequisites & Hardware

You should already be comfortable with:

- Configuring a smart pin (WRPIN / WXPIN / WYPIN, raising DIR to enable it).
- Starting code in another cog with COGINIT (the streaming recipes run a small PASM2 engine in its own cog).
- The P2's DAC modes at the mechanism level. This note *uses* the DAC; it does not re-teach it. For how the 8-bit DAC, the dither modes, the four output configurations, and the voltage math work, see the **I/O & Smart Pins User Guide**, Chapter 10 (DAC Output) and §18.3 (DAC noise) — a companion P2 Knowledge Base publication, and the mechanism this note applies.

Hardware:

- A P2 Edge or P2 Eval board.
- **A way to hear or see the output.** For audio, a P2 audio add-on or a simple series-capacitor coupling into a headphone or amplifier input. For a scope, a small RC low-pass (for example 1 kΩ and 10 nF) from the pin smooths the dither into a clean trace. Recipe 4 also uses a second pin as an analog input.
- Nothing else is required to *run* the code. Bench instruments come in only when you want to *characterize* audio quality (signal-to-noise, distortion, effective bits) rather than confirm the build works.

The Idea

A DAC turns a number into a voltage. The P2's per-pin DAC is physically an **8-bit** converter, but the smart pin reaches an effective **16-bit** resolution by *dithering* — rapidly switching the 8-bit DAC between two adjacent levels so that, averaged over time, the output sits at an in-between voltage. An RC low-pass on the pin (or simply the bandwidth of a speaker) does the averaging. So the mental model is:

> You hand the smart pin a 16-bit number once per sample period; it dithers the 8-bit DAC to approximate that level, and the analog filter turns the stream of levels into a smooth, continuously varying voltage.

That single idea — *a number per period becomes a voltage* — is the foundation under every recipe here. Three consequences are worth holding onto before any detail:

- **The smart pin sets the sample clock.** You program a sample period in clocks; the pin raises its IN flag at the end of each period to say "I captured your number — give me the next one." Your job is to have the next sample ready when that happens.
- **A continuous signal is just the right sequence of numbers.** A tone, a recorded sound, a swept waveform — each is a stream of 16-bit values delivered one per period. Generating that stream is the whole task; the DAC mode never changes.
- **Frequency is decoupled from the sample rate by a phase accumulator.** To play a waveform at an arbitrary pitch, you step a 32-bit *phase* by a fixed increment each sample and read the waveform at that phase. This is direct digital synthesis (DDS), and it is the shared engine behind playback, synthesis, and mixing.

How It Works (in brief)

The mechanism lives in the **I/O & Smart Pins User Guide**, Chapter 10. A few points from there are all you need to follow the builds.

The output stage. A DAC smart pin is configured with three fields ORed together: the **dither mode**, the **output configuration** (drive impedance and peak voltage), and `P_OE` to enable the output. The builds here use 16-bit PWM dither at the 124-ohm / 3.3 V configuration:

```
WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
WXPIN(DAC_PIN, $100)        ' sample period: 256 clocks
PINHIGH(DAC_PIN)            ' enable the smart pin first
WYPIN(DAC_PIN, $8000)       ' then write the first value (mid-scale)
```

The P2 offers four output configurations — `P_DAC_990R_3V`, `P_DAC_600R_2V`, `P_DAC_124R_3V`, and `P_DAC_75R_2V` — trading drive impedance against peak voltage (3.3 V or 2.0 V). Pick by what drives your load; the User Guide §10.1–10.2 gives the full table.

The two dither modes. `P_DAC_DITHER_PWM` (mode `%00011`) dithers with a regular PWM pattern: it has the cleanest spectrum and the lowest switching noise, but it adds a small spectral component at sysclock/256 (about −48 dB), and its sample period **must be a multiple of 256 clocks**. `P_DAC_DITHER_RND` (mode `%00010`) dithers pseudo-randomly: it has no periodic component but slightly more broadband noise, and with its X period set to 1 it updates every clock. Use PWM dither for audio, PRNG dither for control voltages. (User Guide §10.4; the plain 8-bit noise mode `%00001` is in §18.3.)

Voltage math. A 16-bit code maps to a DC level by **V = (code / 65536) × Vfs**, where Vfs is the configuration's peak (3.3 V here). So `$8000` is mid-scale (about 1.65 V) and `$4000` is a quarter scale (about 0.825 V). This is the relation the User Guide gives in §10.7/§10.10, and the Verify section uses it as a known-answer check.

The sample clock and IN-flag sync. The period you write with WXPIN sets the sample rate: at 200 MHz a 256-clock period gives `200_000_000 / 256 = 781_250` samples per second, well above the audio band, with the PWM-dither tone landing at 781.25 kHz where an RC filter removes it. The smart pin raises its IN flag once per period. The streaming engines select that edge as event 1 and wait on it, so each sample lands exactly on a period boundary:

```
MOV     pa, #DAC_PIN
OR      pa, #%001_000000   ' SE1 = DAC pin's IN rising
SETSE1  pa
' ... each period:
WAITSE1                     ' wait for the next period
WYPIN   smp, #DAC_PIN       ' deliver the next sample
```

DDS in one line. To advance through a waveform at frequency *f*, the phase increment is **inc = f × 2^{32} / sample_rate**. In Spin2 that is the `frac` operator: `inc := f frac sample_rate`. Adding `inc` to a 32-bit phase each sample walks one cycle every `2^32 / inc` samples; the top bits of the phase index a table, or feed the CORDIC directly.

Where the *absolute* accuracy and the audio quality of the output stand — signal-to-noise, distortion, the true effective bit depth — is bounded by the analog front end and the filtering, and those numbers come from a bench measurement, not from the code. This note ships qualitative claims and the known-answer DC check now, and defers the measured figures (see Verify and Pitfalls).

Choosing a Technique

The output stage above is common to everything. The rest of this note is a small catalog of ways to *generate* the sample stream, and a few questions tell you which to reach for:

- **Stored or computed?** Are you playing back recorded sample data, or generating a waveform mathematically?
- **One signal or many?** A single voice, or several mixed together — and to one pin or a stereo pair?
- **Synthesized or live?** Is the signal computed on the P2, or is it an analog input passing through?

Every recipe below is built on the same output stage and the same DDS phase accumulator; each changes only what it must.

If you need...	Use	What it adds
To play recorded sample data at any pitch	**Recipe 1** — sample playback	a phase-accumulator resampler
A computed tone or waveform	**Recipe 2** — waveform synthesis	sine, sawtooth, and triangle from the phase
The cleanest analog from the 8-bit DAC	**Recipe 3** — dithering	the PRNG-vs-PWM choice and an RC filter
To pass a live analog signal through the chip	**Recipe 4** — ADC → DAC passthrough	a sample pump tied to the ADC
Several voices on one or two pins	**Recipe 5** — mixing & panning	software summing and stereo placement
Gapless, hardware-paced playback	**Going Further** — the streamer	a pointer to the Streamer Programming Guide
A full 32-stream audio engine	**The ceiling** — reSound	a reference design, not a starting point

Recipe 1 — Sample Playback (DDS)

Use it when you have stored sample data — a recorded sound, a wavetable, a loop — and want to play it at any pitch.

A phase accumulator resamples the stored buffer: each output sample, the top bits of the phase index the buffer, and the phase advances by an increment that sets how fast the buffer is replayed. Because the increment is independent of the buffer length, the *same* data plays at any pitch. Here a PASM2 cog does the per-sample work, paced by the DAC's IN event; Spin2 fills the buffer once at startup (with synthesized data, so the example is self-contained — in a real program this is recorded or loaded audio).

```
CON
  _clkfreq = 200_000_000

  DAC_PIN   = 48                ' DAC output pin (any free pin)
  PERIOD    = 256               ' DAC sample period in clocks
                                '   (multiple of 256 for PWM dither)
  SAMPLE_HZ = _clkfreq / PERIOD ' resulting sample rate: 781_250 Hz

  TABLE_LEN = 256               ' stored sample buffer length (power of 2)
  PLAY_HZ   = 220               ' how often the buffer repeats, in Hz

VAR
  LONG buffer[TABLE_LEN]        ' stored sample data, filled at startup
  LONG inc                      ' DDS phase increment (the cog reads it)

PUB main() | i, ang
```

continues on next page →

↪ continued from previous page

```
  ' Fill the buffer with one cycle of a two-harmonic wave. In a real
  ' program this is recorded or loaded audio; here we synthesize it so
  ' the example is self-contained. The cog resamples whatever is in the
  ' buffer to the requested pitch.
  ang := 0
  repeat i from 0 to TABLE_LEN-1
    buffer[i] := qsin($6000, ang, TABLE_LEN) + qsin($1800, ang*3, TABLE_LEN)
    ang++

  ' DDS phase step = PLAY_HZ * 2^32 / SAMPLE_HZ, so the whole buffer
  ' repeats PLAY_HZ times per second at our DAC sample rate.
  inc := PLAY_HZ frac SAMPLE_HZ

  COGINIT(NEWCOG, @player, @buffer)        ' load+run the player below

DAT             ORG

player          MOV     bufp, ptra            ' @buffer (sample base)
                MOV     incp, ptra
                ADD     incp, ##(TABLE_LEN*4) ' @inc follows the buffer
                RDLONG  pinc, incp

                ' configure the DAC output stage (foundation: IOSP Ch.10)
                WRPIN   dacmode, #DAC_PIN
                WXPIN   #PERIOD, #DAC_PIN
                DIRH    #DAC_PIN              ' enable the smart pin
                WYPIN   ##$8000, #DAC_PIN     ' park at mid-scale

                ' event 1 = DAC pin's IN rising, once per sample period
                MOV     pa, #DAC_PIN
                OR      pa, #%001_000000
                SETSE1  pa

.loop           MOV     idx, phase            ' top 8 bits = buffer index
                SHR     idx, #24              ' TABLE_LEN=256 -> >> (32-8)
                SHL     idx, #2               ' long index -> byte offset
                ADD     idx, bufp             ' hub address of buffer[idx]
                RDLONG  smp, idx              ' fetch the stored sample
                ADD     smp, ##$8000          ' signed -> offset binary
                WAITSE1                        ' wait for next sample period
                WYPIN   smp, #DAC_PIN         ' hand the DAC its sample
                ADD     phase, pinc           ' advance the DDS phase
                JMP     #.loop

dacmode         LONG    P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE
phase           LONG    0

bufp            RES     1
incp            RES     1
pinc            RES     1
idx             RES     1
smp             RES     1
```

How this works. Spin2 fills a 256-entry buffer, then computes one number — `inc`, the DDS phase increment — and launches the player cog with a pointer to the buffer. The cog reads the increment (it lives in hub right after the buffer), configures the DAC, and selects the DAC pin's IN-rising edge as its sample-clock event. Each period it takes the top eight bits of the 32-bit phase as a buffer index, fetches that sample, offsets it from signed to the DAC's offset-binary range (adding `$8000`), waits for the period boundary, and delivers it — then advances the phase. To change the pitch you change only `PLAY_HZ`; to play different audio you change only the buffer contents.

★ **Tip:** The index uses the *top* bits of the phase, not the bottom, so fractional phase positions between buffer entries are simply truncated. That is good enough for most playback; for cleaner resampling you would interpolate between adjacent entries using the discarded low bits.

♦ **Verify:** With an RC filter and a scope on pin 48, you see a repeating two-harmonic waveform at 220 Hz; on a speaker it is a steady low tone. Change `PLAY_HZ` to 440 and the pitch rises an octave with no other change. If the output is silent, confirm the pin and that DEBUG is not required here (this recipe uses no `DEBUG()`); if it is distorted, the buffer values may be exceeding the signed 16-bit range — keep the synthesized amplitudes summing within `±$7FFF`.

Recipe 2 — Waveform Synthesis (DDS)

Use it when you want a tone or a waveform computed on the fly — an oscillator, an LFO, a test signal — with no stored data.

The same phase accumulator drives a *computed* waveform instead of a table lookup. A CORDIC QROTATE turns the phase directly into a sine; a sawtooth is just the top bits of the phase; a triangle is the sawtooth folded at the half-cycle. One constant, `WAVE`, selects which.

```
CON
  _clkfreq = 200_000_000

  DAC_PIN   = 48
  PERIOD    = 256                ' DAC sample period (multiple of 256)
  SAMPLE_HZ = _clkfreq / PERIOD  ' 781_250 Hz sample rate

  TONE_HZ   = 440                ' output frequency (A4)
  WAVE      = 0                  ' 0 = sine, 1 = sawtooth, 2 = triangle

VAR
  LONG inc

PUB main()
  inc := TONE_HZ frac SAMPLE_HZ  ' DDS step = TONE_HZ * 2^32 / SAMPLE_HZ
  COGINIT(NEWCOG, @synth, @inc)

DAT             ORG

synth           RDLONG  pinc, ptra            ' DDS phase increment
```

continues on next page →

↪ continued from previous page

```
                ' configure the DAC output stage (foundation: IOSP Ch.10)
                WRPIN   dacmode, #DAC_PIN
                WXPIN   #PERIOD, #DAC_PIN
                DIRH    #DAC_PIN
                WYPIN   ##$8000, #DAC_PIN

                MOV     pa, #DAC_PIN
                OR      pa, #%001_000000      ' SE1 = DAC pin IN rising
                SETSE1  pa

.loop           ADD     phase, pinc           ' advance the DDS phase
                CMP     wsel, #1        wz
        if_e    JMP     #.saw
                CMP     wsel, #2        wz
        if_e    JMP     #.tri

.sine           QROTATE ampl, phase           ' (AMPL, phase) -> (X, Y)
                GETQY   smp                   ' Y = AMPL * sin(phase)
                ADD     smp, ##$8000          ' signed -> offset binary
                JMP     #.emit

.saw            MOV     smp, phase            ' rising ramp from the phase
                SHR     smp, #16              ' top 16 bits: 0..$FFFF
                JMP     #.emit

.tri            MOV     smp, phase            ' fold the ramp to a triangle
                SHL     smp, #1
                TEST    phase, hibit    wc    ' second half of the cycle?
        if_c    NOT     smp                   '   -> count back down
                SHR     smp, #16

.emit           WAITSE1                        ' wait for next sample period
                WYPIN   smp, #DAC_PIN
                JMP     #.loop

dacmode         LONG    P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE
ampl            LONG    $7FFF
wsel            LONG    WAVE
hibit           LONG    $8000_0000
phase           LONG    0

pinc            RES     1
smp             RES     1
```

How this works. Each period the cog advances the phase, then branches on `WAVE`. For the **sine**, QROTATE rotates the vector `(AMPL, 0)` by the phase angle and GETQY returns `AMPL * sin(phase)` — the same one-instruction trig the CORDIC note (P2AN002) builds on — which is then offset to the DAC's range. For the **sawtooth**, the top 16 bits of the phase *are* a rising ramp, already in range. For the **triangle**, doubling the phase and inverting it on the second half of the cycle folds the ramp into a symmetric up-and-down. Frequency is set entirely by `TONE_HZ` through the phase increment; the waveform shape is independent of it.

★ **Tip:** Because the angle is the full 32-bit phase, the sine needs no table and no wrap handling — the phase overflow *is* the cycle boundary. The same QROTATE gives you a cosine (GETQX) on the same phase, which is the quadrature pair behind more advanced modulation.

♦ **Verify:** On a scope through an RC filter, `WAVE = 0` traces a clean 440 Hz sine, `WAVE = 1` a sawtooth, and `WAVE = 2` a triangle, each peaking near the rails. On a speaker, the three settings have the characteristic sine/saw/triangle timbres at the same pitch. A sine that looks like a triangle means the RC filter is cutting too low; a stair-stepped trace means it is cutting too high to smooth the dither.

Recipe 3 — Dithering for Effective Resolution

Use it when you care about the analog quality of the output — the smoothness of a control voltage, the noise floor of an audio signal — and need to choose the dither mode and filter deliberately.

The DAC is physically 8 bits; both 16-bit modes reach finer resolution by dithering between adjacent 8-bit levels and letting an external filter average the result. The choice between them is a real trade, and so is the RC filter that follows. This program walks the full 16-bit code range in small steps so a filtered scope can show the settled level at each code; switch `DITHER` and `PERIOD` to compare the two modes on your own bench.

```
CON
  _clkfreq = 200_000_000

  DAC_PIN = 48

  ' Choose the 16-bit dither mode for the output stage:
  '   PWM dither  (P_DAC_DITHER_PWM, mode %00011): cleanest spectrum,
  '     lowest switching noise -- but adds a small tone at clock/256;
  '     its X period MUST be a multiple of 256. Best for audio.
  '   PRNG dither (P_DAC_DITHER_RND, mode %00010): no periodic tone,
  '     slightly more broadband noise; X = 1 updates every clock. Best
  '     for control DC.
  DITHER  = P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE
  PERIOD  = $100                ' 256 for PWM dither; use 1 for PRNG

PUB main() | code
  PINCLEAR(DAC_PIN)
  WRPIN(DAC_PIN, DITHER)
  WXPIN(DAC_PIN, PERIOD)
  PINHIGH(DAC_PIN)             ' enable the smart pin first
  WYPIN(DAC_PIN, $8000)        ' then park at mid-scale

  ' Walk the full 16-bit code range in small steps. Codes that fall
  ' between the 8-bit DAC's native levels are produced purely by
  ' dithering, so a filtered scope trace climbs in smooth fine steps.
  repeat
    repeat code from 0 to $FFFF step $0400
      WYPIN(DAC_PIN, code)
      waitms(2)
```

How this works. The program is a slow staircase, so the timing-critical streaming engine of the other recipes is not needed — each level holds long enough for the dither and the RC filter to settle. The interesting work is the configuration. `P_DAC_DITHER_PWM` produces a clean, predictable spectrum but a small fixed tone at sysclock/256, and constrains the period to multiples of 256; `P_DAC_DITHER_RND` removes that tone at the cost of a little more broadband noise and runs with a period of 1. The codes that fall *between* the 8-bit DAC's 256 native levels exist only because the dither produces them — that is where the extra bits live, and they appear only after the analog filter has averaged the switching.

■ **Hardware:** The effective resolution you actually realize is set by the RC filter and the sample rate, not by the "16-bit" label. A filter that cuts too high leaves dither noise on the output; one that cuts too low rounds off your signal. Match the filter's corner to the bandwidth you need: well above the highest signal frequency, well below the dither frequency.

♦ **Verify:** With an RC filter and a scope, the staircase climbs in smooth, even steps far finer than the 256 steps an undithered 8-bit DAC would show. Swap to PRNG dither (`DITHER` to `P_DAC_DITHER_RND`, `PERIOD` to 1) and the steps remain but the noise character changes — grainier, with no periodic tone. The numeric effective-bit, signal-to-noise, and distortion figures depend on your filter and bench setup; measure them on hardware rather than reading a number here (see Pitfalls).

Recipe 4 — Real-Time ADC → DAC Passthrough

Use it when you want a live analog signal to travel through the P2 — a software-defined filter, an effect, or just a buffered "analog wire" — read on one pin and reconstructed on another.

This ties the DAC to the P2's ADC (the subject of application note P2AN001). One cog reads a pin's ADC every sample period and immediately re-emits the value on a DAC pin. The two pins share a 256-clock period, and the ADC's IN event paces the loop, so the pump runs lockstep with the converters.

```
CON
  _clkfreq = 200_000_000

  ADC_PIN = 32                  ' analog input pin (feed a signal here)
  DAC_PIN = 48                  ' analog output pin (the reconstruction)
  PERIOD  = 256                 ' shared sample period in clocks

PUB main()
  COGINIT(NEWCOG, @pump, 0)

DAT             ORG

pump            ' ADC input pin: SINC2 filtering, 256-clock period
                FLTL    #ADC_PIN
                WRPIN   adcmode, #ADC_PIN
                WXPIN   #%01_1000, #ADC_PIN   ' SINC2, 2^8 = 256 clocks
                DIRH    #ADC_PIN
```

continues on next page →

↪ continued from previous page

```
                ' DAC output pin: 16-bit PWM dither, matched period
                WRPIN   dacmode, #DAC_PIN
                WXPIN   #PERIOD, #DAC_PIN
                DIRH    #DAC_PIN
                WYPIN   ##$8000, #DAC_PIN     ' park at mid-scale

                ' SE1 = ADC pin IN rising: fresh accumulator each period
                MOV     pa, #ADC_PIN
                OR      pa, #%001_000000
                SETSE1  pa

                WAITSE1                        ' first SINC2 period settles
                RDPIN   last, #ADC_PIN        ' prime the differencer

.loop           WAITSE1                        ' wait for next ADC period
                RDPIN   acc, #ADC_PIN         ' running SINC2 accumulator
                MOV     smp, acc
                SUB     smp, last             ' delta over this period
                MOV     last, acc             ' remember for next time
                ZEROX   smp, #26              ' true 27-bit modular delta
                ' a 256-clock SINC2 delta is already ~16-bit full scale
                ' (raw width = 2*log2(256)), so it drives the DAC directly
                CMP     smp, ##$FFFF    wcz
        if_a    MOV     smp, ##$FFFF          ' clamp to full scale
                WYPIN   smp, #DAC_PIN         ' re-emit on the DAC
                JMP     #.loop

adcmode         LONG    P_ADC_1X | P_ADC
dacmode         LONG    P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE
last            LONG    0

acc             RES     1
smp             RES     1
```

How this works. The ADC pin runs in SINC2 filtering mode, which maintains a running accumulator; the value over one period is the **difference** between two successive reads (the same differential the instrumentation-ADC note uses). At a 256-clock period that raw differential is already a full-scale 16-bit value — the SINC2 filter's raw width is twice the log of the sample count, so 256 clocks lands right at 16 bits — so the cog feeds it straight to the DAC, clamping to full scale only as a guard. (It primes the differencer with one discarded read first, because a SINC2 difference is valid only from the second period.) That is the whole pump. Feed an analog signal into pin 32 and a reconstructed copy appears on pin 48. The ADC mechanism is owned by the I/O & Smart Pins User Guide, Chapter 16 (and walked in detail in P2AN001); here it is just the source end of the pump.

▲ **Pitfall:** The ADC and DAC must run at the *same* sample period, or the pump under- or over-runs. Both are set to 256 clocks here; if you change one, change the other, and keep the DAC period a multiple of 256 for PWM dither.

♦ **Verify:** The cleanest check is the Tier-0 known answer below — drive the ADC pin to a fixed DC and confirm the DAC reproduces it. With a signal generator on the input and a scope on the output,

the output follows the input up to the band the 256-clock rate and the RC filter pass. Silence or a stuck output usually means the input pin is floating — confirm the source is actually connected.

Recipe 5 — Multi-Channel Mixing & Panning

Use it when you want more than one voice — chords, layered sound effects, stereo — summed in software and placed across one or more output pins.

Mixing on the P2 is exactly what it is on paper: synthesize each voice, scale it by a volume, and add the voices together. Panning is the same sum done twice with different weights, once for a left pin and once for a right. This cog runs three voices — a C-major chord — each with its own DDS phase accumulator and stereo position, sums them, clamps the result, and drives two DAC pins.

```
CON
  _clkfreq = 200_000_000

  LEFT_PIN  = 48                ' left-channel DAC pin
  RIGHT_PIN = 49                ' right-channel DAC pin
  PERIOD    = 256               ' DAC sample period (multiple of 256)
  SAMPLE_HZ = _clkfreq / PERIOD ' 781_250 Hz

VAR
  LONG inc[3]                   ' the voices' DDS phase increments

PUB main()
  inc[0] := 262 frac SAMPLE_HZ  ' C4
  inc[1] := 330 frac SAMPLE_HZ  ' E4
  inc[2] := 392 frac SAMPLE_HZ  ' G4
  COGINIT(NEWCOG, @mixer, @inc)

DAT             ORG

mixer           RDLONG  inc0, ptra[0]         ' per-voice phase steps
                RDLONG  inc1, ptra[1]
                RDLONG  inc2, ptra[2]

                ' two DAC pins: left and right, 16-bit PWM dither
                WRPIN   dacmode, #LEFT_PIN
                WXPIN   #PERIOD, #LEFT_PIN
                DIRH    #LEFT_PIN
                WYPIN   ##$8000, #LEFT_PIN
                WRPIN   dacmode, #RIGHT_PIN
                WXPIN   #PERIOD, #RIGHT_PIN
                DIRH    #RIGHT_PIN
                WYPIN   ##$8000, #RIGHT_PIN

                ' pace the mix on the left pin's IN, once per period
                MOV     pa, #LEFT_PIN
                OR      pa, #%001_000000
                SETSE1  pa
```

continues on next page →

↪ continued from previous page

```
.loop           MOV     lsum, #0              ' start an empty mix
                MOV     rsum, #0

                ADD     pha0, inc0            ' voice 0 (center)
                QROTATE amp, pha0
                GETQY   smp
                MOV     pl, v0panL
                MOV     pr, v0panR
                CALL    #addvoice

                ADD     pha1, inc1            ' voice 1 (panned left)
                QROTATE amp, pha1
                GETQY   smp
                MOV     pl, v1panL
                MOV     pr, v1panR
                CALL    #addvoice

                ADD     pha2, inc2            ' voice 2 (panned right)
                QROTATE amp, pha2
                GETQY   smp
                MOV     pl, v2panL
                MOV     pr, v2panR
                CALL    #addvoice

                FGES    lsum, neg7fff         ' clamp mix to 16-bit signed
                FLES    lsum, pos7fff
                FGES    rsum, neg7fff
                FLES    rsum, pos7fff
                ADD     lsum, ##$8000         ' signed -> offset binary
                ADD     rsum, ##$8000

                WAITSE1                        ' wait for sample period
                WYPIN   lsum, #LEFT_PIN
                WYPIN   rsum, #RIGHT_PIN
                JMP     #.loop

' Scale one voice's sample by its left/right pan weights (0..256) and
' add it into the running stereo mix.
addvoice        MOV     t, smp
                MULS    t, pl
                SAR     t, #8
                ADD     lsum, t
                MOV     t, smp
                MULS    t, pr
                SAR     t, #8
        _ret_   ADD     rsum, t

dacmode         LONG    P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE
amp             LONG    $2AAA                 ' per-voice amplitude
v0panL          LONG    192
v0panR          LONG    192
v1panL          LONG    256
v1panR          LONG     64
v2panL          LONG     64
```

↪ *continued from previous page*

```
v2panR          LONG    256
pos7fff         LONG    $7FFF
neg7fff         LONG    -$7FFF
pha0            LONG    0
pha1            LONG    0
pha2            LONG    0

inc0            RES     1
inc1            RES     1
inc2            RES     1
lsum            RES     1
rsum            RES     1
smp             RES     1
pl              RES     1
pr              RES     1
t               RES     1
```

How this works. Each period the cog clears the left and right accumulators, then for each voice advances its phase, takes a CORDIC sine, and adds it into *both* channels through `addvoice` — scaled by that voice's left and right pan weights (0–256, a fixed-point fraction with the divide done by `SAR #8`). The three per-voice amplitudes are set to about a third of full scale so the sum stays in range; the FGES/FLES pair clamps the mix to signed 16-bit as a guard before the offset and output. Voice 0 sits centered, voice 1 is panned left, voice 2 right — so on headphones the chord spreads across the stereo field.

★ **Tip:** Volume and pan are the same operation — a multiply by a weight. To fade a voice, lower both its weights together; to move it in the stereo field, shift weight from one side to the other while keeping the sum constant. Adding a fourth voice is one more `addvoice` block.

♦ **Verify:** On a stereo output you hear a C-major chord with the three notes placed left, center, and right. Scope either pin and you see the summed waveform; scope both and the left/right difference reflects the panning. If the mix sounds clipped, your per-voice amplitudes sum past `$7FFF` — lower `amp` or the pan weights.

The Ceiling — A 32-Stream Software Mixer

The recipes above scale to a handful of voices in one cog. To see how far the idea goes, look at **reSound** (Johannes Ahlebrand, **OBEX #2861**) — a single-cog audio engine that mixes **up to 32 input streams**, each with independent volume, panning, pitch, and sample format (8- or 16-bit, signed or unsigned), to anywhere from one to **eight** output pins for mono, stereo, or surround. Its companion MOD-style music player turns tracker files into audio the same way.

reSound is built from exactly the primitives in this note — the dithered DAC output stage, a per-channel DDS phase accumulator for pitch, software summing, and multi-pin fan-out — generalized and optimized into a full engine of roughly a thousand lines. This note does not rebuild it; it is a *reference* for the ceiling, not a starting point. If you have followed Recipes 1 and 5, you have the foundation reSound is built on, and its source (cited in the Resources) will read as a scaled-up version of what you already understand.

Going Further

Streamer-fed playback. The recipes here pace the DAC from a cog, one sample per period. For gapless playback of long buffers — or for sample rates where per-sample cog work would be tight — the P2's **streamer** can feed the DAC directly from hub memory with no per-sample software at all, freeing the cog entirely. The streamer is its own subsystem with its own timing and command set; that composition belongs to the **Streamer Programming Guide**, a companion P2 Knowledge Base publication, which owns the streamer mechanics this note would otherwise duplicate. Use this note for generating and mixing samples, that guide for streaming them out at high, rock-steady rates.

When you don't need the DAC at all. For a simple beep or a monotonic tone into a piezo, you do not need the DAC: a smart pin in NCO-frequency mode drives a square-wave tone on a pin directly (the technique behind community tone players such as EZ Sound, OBEX #2860). That is the NCO-frequency story in the User Guide's Chapter 8, not a DAC technique — reach for it when a square wave is all you need and the analog quality of the DAC would be wasted.

See It Work / Verify

Every recipe shares the same foundation, so one verification approach covers them all — and the strongest check needs no instruments.

The Tier-0 known answer. A fixed 16-bit DAC code produces a computable DC voltage, **V = (code / 65536) × Vfs**. Drive the DAC to that code, jumper the pin to a free pin running the P2AN001 instrumentation ADC, and the ADC must read back that voltage (within the front-end tolerance) — proving the whole signal path with one jumper and no bench gear. This short program drives the fixed level and prints the value the ADC should report:

```
CON
  _clkfreq = 200_000_000

  DAC_PIN = 48
  VFS_UV  = 3_300_000           ' full scale for P_DAC_124R_3V (3.3 V)
  CODE    = $4000               ' a fixed 16-bit DAC code (quarter scale)

PUB main() | expect_uV
  ' Drive a fixed, known DC level on the DAC.
  PINCLEAR(DAC_PIN)
  WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
  WXPIN(DAC_PIN, $100)
  PINHIGH(DAC_PIN)              ' enable the smart pin first
  WYPIN(DAC_PIN, CODE)         ' then write the fixed code

  ' The DC this code should produce: V = (code / 65536) * Vfs.
  expect_uV := muldiv64(CODE, VFS_UV, $1_0000)
  DEBUG("code ", uhex_long(CODE), " -> expect ", udec(expect_uV), " uV")
```

With `CODE = $4000` the program reports **825000 µV** (0.825 V, a quarter of 3.3 V), and a P2AN001 ADC on the jumpered pin should read close to it. Build with DEBUG enabled (`pnut_ts -d`) so the printed line appears.

What success looks like for the streaming recipes. Through an RC filter on a scope, each recipe shows its expected shape — a repeating buffer (R1), the selected waveform (R2), a fine staircase (R3), the input signal reconstructed (R4), or the summed chord (R5). On a speaker, the audio recipes sound as described.

When it doesn't work:

- **No output at all.** The smart pin is not enabled, or DIR was raised before WRPIN/WXPIN configured it — configure the mode first, then enable. Confirm the output pin and that `P_OE` is in the mode word.
- **A buzzing or harsh tone where you expected a clean one.** The RC filter is not removing the dither (corner too high), or for PWM dither you are hearing the sysclock/256 component — filter below it.
- **A muffled or rounded waveform.** The RC filter corner is too low and is attenuating the signal itself; raise it.
- **Distortion or clipping in the mix (R5).** The summed voices exceed signed 16-bit; lower the per-voice amplitude or pan weights.
- **Nothing printed by the Tier-0 program.** Build with DEBUG enabled (`pnut_ts -d`); without it the `DEBUG()` directive is ignored.

What is deferred to hardware. Numeric audio-quality figures — signal-to-noise ratio, total harmonic distortion, the true effective bit depth after filtering — depend on the analog filter and the measurement bench and are **not** asserted here. They come from a characterization run on real silicon, exactly as the instrumentation-ADC note defers its ENOB figure; until then this note ships qualitative behavior and the Tier-0 DC known answer.

Pitfalls & Notes

▲ **Pitfall — write the mode before enabling the pin.** Configure the smart pin with WRPIN and WXPIN *before* raising DIR, and write the first value with WYPIN *after*. The order in the output-stage snippet (mode, period, enable, value) is the safe one; reversing it leaves the first output cycle undefined.

▲ **Pitfall — PWM dither needs a period that is a multiple of 256.** `P_DAC_DITHER_PWM` requires its X period to be a multiple of 256 clocks (256 is the minimum). A period that is not produces incorrect dithering. If you need a period that is not a multiple of 256, use `P_DAC_DITHER_RND` instead.

■ **Hardware — always filter the output.** The DAC's analog value only emerges after the dither is averaged. A simple RC low-pass on the pin does this; choose its corner above your highest signal frequency and below the dither frequency (sysclock/256 for PWM dither). Without a filter you are looking at the raw switching, not the intended signal.

■ **Hardware — pick the output configuration for your load.** The four configurations trade drive impedance against peak voltage: lower impedance drives heavier loads (headphones, line inputs) but the User Guide §10.1–10.2 has the exact figures. Match the configuration to what the pin must drive, and remember the peak is 3.3 V or 2.0 V depending on the choice — the voltage math scales with it.

★ **Tip — the sample rate is your headroom.** A 256-clock period at 200 MHz gives 781 kHz, far above audio, so there is plenty of room. Raising the period lowers the sample rate, though the PWM dither tone stays fixed at sysclock/256 regardless of the period; lowering it is bounded by 256 for PWM dither. There is no need to overclock — the audio band sits comfortably inside the available rate.

♦ **Verify — the loopback proves the code, not the fidelity.** The Tier-0 DAC-to-ADC check confirms the signal path and the arithmetic, but because both converters share the same die they share imperfections; it is a functional check, not an accuracy or audio-quality benchmark. Characterize fidelity against external instruments.

Conclusion

The idea under every recipe in this note is the same one: a P2 pin DAC turns a 16-bit number into a voltage, once per sample period, and a continuous analog signal is just the right sequence of those numbers delivered on time. Build the output stage once — choose a dither mode, set a sample period, sync to the pin's IN flag — and everything else is a way to generate the stream: read it from a buffer, compute it from a phase, pass it through from the ADC, or sum several streams into one. The DDS phase accumulator decouples pitch from the sample rate, the dither modes trade spectrum for noise, and a single multiply-and-add is all that mixing and panning require. The 32-stream reSound engine is this same handful of primitives pushed to its limit. Start from the output stage, pick the recipe your project needs, and confirm it with the one-jumper known-answer check before you trust a single sample.

Resources

- **Example code (download).** The five worked builds in this note — sample playback, waveform synthesis, dithering, ADC passthrough, and the stereo mixer — are published together as the **P2AN003 source ZIP** beside this note's PDF, each a complete `pnut_ts`-compiled program ready to build and run.
- **P2 I/O & Smart Pins User Guide, Chapter 10 (DAC Output)** and **§18.3 (DAC noise)** (a companion P2 Knowledge Base publication) — the mechanism this note applies: the 8-bit DAC, the dither modes, the four output configurations, the voltage math, the IN-flag sample sync, and ADC feedback during DAC output. Chapter 16 (ADC) and application note **P2AN001** cover the input end used in Recipe 4.
- **P2 Streamer Programming Guide** (a companion P2 Knowledge Base publication) — for feeding the DAC from hub memory at high, hardware-paced sample rates (the "Going Further" path).
- **OBEX #2861 — reSound** (Johannes Ahlebrand): the 32-stream, up-to-8-pin software audio mixer and MOD player behind The Ceiling. Find it by object number at the Parallax OBEX (`obex.parallax.com`).

References

These are the authoritative primary sources the note's facts trace to. (The companion *P2 I/O & Smart Pins User Guide* named in Resources derives from these same sources.)

1. *Propeller 2 Documentation v35 — Rev B/C Silicon* (Chip Gracey, Parallax Inc.) — Smart Pins, DAC modes `%00001`/`%00010`/`%00011`: the 8-bit DAC and its dithering, the four output configurations, the sample-period and IN-flag behavior.
2. *Spin2 Language Documentation* (Chip Gracey, Parallax Inc.) — smart-pin configuration (`WRPIN`/`WXPIN`/`WYPIN`), `QSIN`/QROTATE, `MULDIV64`, and the `frac` operator used for the DDS phase increment.
3. *Propeller 2 Datasheet* (Parallax Inc.) — DAC electrical characteristics and output configurations.

Revision History

Version	Date	Notes
1.0.2	2026-08-08	License restored to CC BY-SA 4.0 (share and adapt, including commercially, with attribution and under the same terms); the Trademarks note now states the license grants permissions under copyright only. No technical content changed.
1.0.1	2026-07-11	DAC precision refinement: the PWM-dither spectral component sits at a fixed sysclock/256, independent of the sample period, so raising the period lowers the sample rate without moving the dither tone. No recipes added.
1.0.0	2026-07-03	First community-review release. Techniques-catalog: shared dithered-DAC output stage + five recipes (sample playback, waveform synthesis, dithering, ADC-to-DAC passthrough, mixing & panning) + a 32-stream reference ceiling (referenced). All embedded and example-library code compiles clean under `pnut_ts` (`-d` where it uses `debug()`); audio-quality figures qualitative pending hardware characterization. Ships a downloadable example library.

Copyright & License

Acknowledgments

- **Parallax Inc.** — for the Propeller 2 and the reference documentation that forms the foundation of this work.
- **Chip Gracey** — for the design of the P2 smart-pin DAC and the silicon and Spin2 documentation that define its behavior.
- **Johannes Ahlebrand** — for reSound (OBEX #2861), the 32-stream software audio mixer that demonstrates these primitives at full scale and seeds the playback and mixing recipes.
- **Michael Mulholland and Chip Gracey** — for the Parallax “Analog Frequency to DAC” and “Digital Sample ADC to DAC” example code, which grounded the synthesis and passthrough recipes.